

Terraform Repository Modularization and Standardization

Overview

The current Terraform implementation works, but as the number of Azure resources and infrastructure components increases, maintaining Terraform code separately for each resource becomes difficult.

The objective of this change is to standardize the Terraform repository structure, introduce reusable modules, centralize common infrastructure logic, and establish proper version management.

The proposed approach will help us:

- Reduce duplicate Terraform code
- Reuse common infrastructure components
- Simplify maintenance
- Maintain consistent implementations across Azure resources
- Manage Terraform and AzureRM provider versions
- Introduce controlled module versioning
- Make future infrastructure onboarding easier

The deployment mechanism using GitHub Actions and Terraform will continue. The main changes will be around repository structure, module management, versioning, and workflow path references.

Current Structure

Currently, Terraform files are maintained at the repository root along with the environment configuration.

```
Repository
├─ main.tf
├─ provider.tf
├─ variables.tf
├─ outputs.tf
├─ data.tf
├─ versions.tf
├─ environments/
│   └─ dev/
│   └─ qa/
│   └─ prod/
└─ .github/workflows/
```

This structure works for the current implementation, but as more Azure resources and infrastructure components are introduced, common Terraform code starts getting repeated across repositories.

Current Pain Areas

Repeated Terraform Code

Common infrastructure components may need to be implemented separately for multiple Azure resources.

For example, if multiple Azure resources require similar networking, storage, Key Vault, monitoring, or compute resources, similar Terraform code may exist in multiple repositories.

This increases maintenance effort.

Common Changes Need to Be Repeated

If a common infrastructure implementation changes, the same change may need to be implemented and tested across multiple Azure resource configurations.

This increases the possibility of differences between implementations.

Provider Version Management

Terraform and AzureRM provider versions can change over time.

Without proper version tracking, different Azure resource configurations may end up using different versions without a clear understanding of compatibility.

Inconsistent Implementations

When each Azure resource maintains its own infrastructure logic, the same type of resource may be implemented differently.

For example, two Azure resource configurations may create similar resources but use different naming conventions, configurations, tags, security settings, or monitoring configurations.

Increasing Maintenance Overhead

As the number of Azure resources increases, the amount of Terraform code that needs to be maintained also increases.

This makes future changes and upgrades more time-consuming.

Proposed Approach

The proposed approach is to standardize the Terraform repository structure and introduce reusable Terraform modules.

The main principles are:

- Keep all Terraform configuration files under a dedicated `terraform/` folder
- Keep environment-specific configuration under the existing root-level `environments/` folder
- Separate environment-specific configuration from common Terraform configuration
- Create reusable Terraform modules for common infrastructure components
- Maintain common modules centrally
- Version modules using tags/releases
- Track Terraform and AzureRM provider compatibility
- Test modules before releasing a new version
- Allow Azure resource configurations to consume a specific module version
- Continue using the existing GitHub Actions deployment mechanism

Proposed Repository Structure

The proposed repository structure is:

```
Repository
├── terraform/
│   ├── main.tf
│   ├── provider.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── data.tf
│   └── versions.tf
├── environments/
│   ├── dev/
│   │   ├── backend.tfvars
│   │   └── terraform.tfvars
│   ├── qa/
│   │   ├── backend.tfvars
│   │   └── terraform.tfvars
│   └── prod/
│       ├── backend.tfvars
│       └── terraform.tfvars
└── .github/workflows/
```

The main change is that all Terraform configuration files are now maintained inside the `terraform/` folder.

The environment-specific configuration remains at the repository root under the `environments/` folder.

Nothing Terraform configuration-related needs to remain directly at the repository root.

Terraform Folder Responsibilities

The `terraform/` folder will contain the Terraform configuration required for the Azure resources.

For example:

```
terraform/  
├─ main.tf  
├─ provider.tf  
├─ variables.tf  
├─ outputs.tf  
├─ data.tf  
└─ versions.tf
```

`main.tf`

Contains the main Terraform resource and module composition.

`provider.tf`

Contains provider configuration.

`variables.tf`

Contains input variables required by the Terraform configuration.

`outputs.tf`

Contains Terraform outputs.

`data.tf`

Contains required Terraform data sources.

`versions.tf`

Contains Terraform and provider version requirements.

Environment Structure

Environment-specific values will remain separated from the common Terraform configuration.

Example:

```
Repository
├── terraform/
│   ├── main.tf
│   ├── provider.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── data.tf
│   └── versions.tf
└── environments/
    ├── dev/
    │   ├── backend.tfvars
    │   └── terraform.tfvars
    ├── qa/
    │   ├── backend.tfvars
    │   └── terraform.tfvars
    └── prod/
        ├── backend.tfvars
        └── terraform.tfvars
```

This allows the same Terraform configuration to be used across multiple environments while passing different values.

For example, the same infrastructure module can be used for DEV, QA, and PROD with different inputs.

Why Modules?

The main reason for introducing modules is to avoid writing the same Terraform implementation multiple times.

A module allows us to build a common infrastructure component once and reuse it across multiple Azure resources.

For example, instead of creating separate Terraform code for a standard networking configuration for every Azure resource, we can create one reusable network module.

The Azure resource configuration can then consume that module by passing the required inputs.

The same approach can be applied to:

- Network
- Compute
- Storage
- Key Vault
- App Service
- Database
- Monitoring
- Security
- Other common infrastructure components

Main Benefit of Modules

The main benefit can be summarized as:

Build once, reuse many times by passing different inputs.

With reusable modules, the same module can be used multiple times by simply passing different inputs.

For example:

```
Azure Resource A
  ↓
Network Module
  ↓
Different inputs

Azure Resource B
  ↓
Network Module
  ↓
Different inputs

Azure Resource C
  ↓
Network Module
```

↓
Different inputs

The underlying infrastructure logic remains centralized, while each Azure resource provides its own configuration.

This reduces duplicate Terraform code and makes common changes easier to manage.

Module Structure

A standard module can follow a structure such as:

```
module-name/  
├─ main.tf  
├─ variables.tf  
├─ outputs.tf  
├─ versions.tf  
└─ README.md
```

`main.tf`

Contains the resources created by the module.

`variables.tf`

Defines the inputs accepted by the module.

`outputs.tf`

Defines values that can be returned to the Azure resource configuration consuming the module.

`versions.tf`

Defines Terraform and provider compatibility requirements.

`README.md`

Documents how the module should be consumed and what inputs and outputs are available.

Terraform Configuration Reuse

The Terraform configuration should focus on composing infrastructure using reusable modules rather than implementing common infrastructure logic repeatedly.

For example:

```
Terraform Configuration
|
+-- Network Module
|
+-- Storage Module
|
+-- Key Vault Module
|
+-- Monitoring Module
```

Each module can be configured using different inputs based on the Azure resource requirements.

This provides a consistent implementation while still allowing resource-specific configuration.

A module can be referenced from the Terraform configuration by passing the required inputs.

For example:

```
module "network" {
  source = "central-module-source/network"
  version = "1.2.0"

  + vnet_name      = var.vnet_name
  + address_space = var.address_space
  + location       = var.location
}
```

The Azure resource configuration can provide different values while reusing the same module implementation.

Centralized Module Repository

The long-term approach is to maintain reusable modules in a centralized repository.

For example:

```
Central Terraform Module Repository
├─ network
├─ compute
├─ storage
├─ keyvault
├─ appservice
├─ database
├─ monitoring
├─ security
└─ snowflake
```

The central repository becomes the common place for maintaining reusable Terraform modules.

Azure resource repositories can consume these modules instead of maintaining their own copy of the same infrastructure logic.

This also makes it easier to manage common changes centrally.

Module Versioning

Modules should be versioned so that Azure resource configurations can consume a known and tested version.

For example:

```
network-module
├─ v1.0.0
├─ v1.1.0
├─ v1.2.0
└─ v2.0.0
```

When a change is made to the module:

1. Make the required change
2. Test the module
3. Update documentation if required
4. Create a new version/tag
5. Make the new version available for consumption

An Azure resource configuration can then reference a specific module version.

For example:

```
version = "1.2.0"
```

This means an Azure resource configuration is not automatically impacted whenever a new module version is released.

The resource team can upgrade to the newer version when they are ready.

This provides controlled upgrades and reduces unexpected impact.

AzureRM Provider and Terraform Version Management

Terraform and AzureRM provider versions should also be tracked as part of the module lifecycle.

Each module should clearly define the supported:

- Terraform version
- AzureRM provider version

For example:

```
Module: Network
Version: 1.2.0

Terraform:
>= 1.x

AzureRM:
>= 4.x
```

The exact versions will depend on the module requirements.

When the AzureRM provider or Terraform version is upgraded, the impact on existing modules should be reviewed and tested.

If changes are required in a module because of a provider upgrade, the module can be updated and released as a new version.

This allows the provider upgrade to be handled centrally rather than requiring every Azure resource configuration to independently identify and fix the same issue.

Version and Module Inventory

As the number of reusable modules increases, maintaining an inventory will help provide visibility into what is available and which versions are currently supported.

The inventory can contain information such as:

Module	Current Version	Terraform Version	AzureRM Version	Owner	Status	Documentation
Network	1.2.0	Supported version	Supported version	Platform Team	Active	Available
Storage	1.1.0	Supported version	Supported version	Platform Team	Active	Available
Key Vault	1.0.0	Supported version	Supported version	Platform Team	Active	Available

This provides a central view of the available modules and their supported versions.

It also makes it easier to identify which module version should be consumed for a particular Azure resource.

Module Testing Before Release

Before a module version is made available for Azure resource consumption, the module should go through validation and testing.

A simple flow can be:

```
Code Change
  ↓
Terraform Format
  ↓
Terraform Validate
  ↓
Plan / Test Deployment
  ↓
Review
  ↓
```

Version Tag



Azure Resource Consumption

The testing approach can be expanded based on the module and its complexity.

The important point is that a module should be tested before releasing a new version.

This helps avoid pushing an untested change to multiple Azure resource configurations.

GitHub Actions Changes

The existing GitHub Actions deployment mechanism can continue to be used.

The main change is the Terraform working directory.

Currently, Terraform commands may run from the repository root:

```
working-directory: .
```

With the proposed structure, Terraform commands should run from:

```
working-directory: terraform
```

For example:

```
terraform init
terraform plan
terraform apply
```

will execute from the `terraform/` directory.

The existing reusable GitHub Actions workflows can continue to be used.

The workflow mainly needs to be updated to point to the new Terraform location.

Environment Path Changes in Workflow

Environment-specific files will remain at the repository root.

Because the Terraform working directory changes to `terraform/`, the workflow needs to explicitly point to the root-level environment paths.

For example, instead of:

```
terraform/environments/dev/backend.tfvars
terraform/environments/dev/terraform.tfvars
```

the workflow should reference:

```
environments/dev/backend.tfvars
environments/dev/terraform.tfvars
```

or use the repository path explicitly where required:

```
$(Build.SourcesDirectory)/environments/dev/backend.tfvars
$(Build.SourcesDirectory)/environments/dev/terraform.tfvars
```

The Terraform commands will still execute from the `terraform/` working directory, while the environment-specific `.tfvars` files continue to be maintained at the repository root.

The deployment flow itself does not need to change.

The main requirement is to make sure all workflow references point to the new Terraform working directory and the existing root-level environment paths correctly.

Impact of the Change

What Changes

- Terraform files move under the `terraform/` folder
- Environment configuration remains under the root-level `environments/` folder
- GitHub Actions Terraform working-directory/path references are updated
- Reusable Terraform modules are introduced
- Common infrastructure logic is centralized
- Module versions are managed

- Terraform and AzureRM provider versions are tracked
- Modules are tested before release

What Does Not Change

- Terraform remains the infrastructure-as-code tool
- Environment separation remains
- GitHub Actions remains the deployment mechanism
- Existing deployment process remains
- Azure resource-specific configuration remains
- The overall infrastructure provisioning approach remains the same

The change is mainly focused on standardization, reuse, maintainability, and centralized management.

Future Scope

The initial implementation can be extended over time.

Potential future improvements include:

Additional Reusable Modules

Create and maintain modules for commonly used infrastructure such as:

- Networking
- Compute
- Storage
- Key Vault
- App Services
- Databases
- Monitoring
- Security
- Snowflake
- Other commonly used services

Centralized Module Repository

Move reusable modules into a dedicated central repository and, where applicable, a private module registry.

Module Inventory

Maintain a central inventory containing:

- Module name
- Module version

- Terraform version
- AzureRM provider version
- Owner
- Status
- Documentation
- Latest release

Terraform and AzureRM Version Tracking

Maintain visibility into supported Terraform and AzureRM versions for each module.

Module Testing

Introduce automated testing and validation before module releases.

Drift Detection

Introduce Terraform drift detection to identify infrastructure changes made outside Terraform.

This can help ensure that the actual infrastructure remains aligned with the Terraform configuration.

Recommended Operating Model

The long-term operating model can follow this approach:

```
Platform / Module Team
    ↓
Develop Module
    ↓
Test Module
    ↓
Review
    ↓
Create Version / Tag
    ↓
Publish Centrally
    ↓
Azure Resource Teams Consume Version
    ↓
Upgrade When Required
```

The platform/module team owns the common infrastructure logic, while Azure resource teams consume the modules based on their resource requirements.

This creates a clear separation between reusable infrastructure implementation and resource-specific configuration.

Summary

The proposed Terraform standardization focuses on moving Terraform configuration into a dedicated `terraform/` folder, introducing reusable modules, centralizing common infrastructure logic, and implementing proper version management.

The environment-specific configuration remains at the repository root, while the Terraform execution working directory changes to the `terraform/` folder.

The GitHub Actions workflow will be updated accordingly so that Terraform commands execute from the `terraform/` directory while environment `.tfvars` files continue to be referenced from the root-level `environments/` directory.

The key approach is:

Build once → Test → Version → Reuse across Azure resources → Upgrade when ready

This approach will reduce duplicate Terraform code, improve consistency, reduce maintenance effort, provide controlled upgrades, and make it easier to onboard additional Azure resources and infrastructure components in the future.